

# AGENTS.md — SIH26025 Mine Subsidence Monitoring Dashboard

## Project Overview

This is the live web dashboard for our Smart India Hackathon submission, problem statement **SIH26025**: an AI-driven underground coal mine subsidence monitoring system using IoT sensor networks. The dashboard visualizes (simulated, demo-grade) sensor data and a hybrid physics-ML subsidence risk score for judges during the final demo.

Domain context the agent must respect when generating copy, labels, or logic:

Sensors: MEMS tiltmeters (tilt angle, °), strain gauges (microstrain,  $\mu\epsilon$ ), geophones (vibration velocity, mm/s)

Networking: Zigbee mesh for short-range node communication, LoRa for long-range backhaul

Satellite layer: InSAR / SBAS-InSAR (Sentinel-1) for surface deformation, referenced conceptually only — not implemented live

Regulatory/reference bodies: DGMS (Directorate General of Mines Safety), CMPDI

Target coalfields: Jharia and Raniganj (Jharkhand / West Bengal)

Mining method: room-and-pillar

Core innovation to convey: hybrid physics-ML model fusing geotechnical parameters with sensor time series to predict subsidence risk, not just detect it after the fact

## Tech Stack (do not deviate without asking)

Next.js 14 App Router, TypeScript, Tailwind CSS

shadcn/ui for components, lucide-react for icons

Recharts for time-series charts

Leaflet + react-leaflet for the coalfield map (no Mapbox API key dependency)

Zustand for global sensor/alert state

No backend/database — this is a frontend demo. All "live" data comes from an in-browser simulator, not a real API.

## Data Simulation Rules

`lib/sensorSimulator.ts` is the single source of truth for fake sensor data.

Readings must look physically plausible: slow baseline drift + small random noise, NOT pure random jitter.

Tiltmeter values should stay within a few degrees; strain gauge in the tens-to-hundreds of  $\mu\epsilon$ ; geophone near-zero with occasional small spikes.

Must expose a `triggerSubsidenceEvent()` function that ramps all readings toward danger thresholds over ~15-20 seconds and flips the overall risk score to "Critical" — this is used live during the judge demo.

Never claim this is real sensor data anywhere in the UI copy. Use language like "Live simulated feed" or "Demo mode" in a small badge if asked, but don't be heavy-handed about it — the point is a convincing demo, not a disclaimer-heavy one.

## Coding Conventions

Functional components only, TypeScript strict mode.

Co-locate types with the feature that owns them (e.g. `types.ts` per feature folder), not one giant global types file.

Tailwind for all styling — no CSS modules, no styled-components.

Keep components under ~150 lines; extract subcomponents rather than growing a file.

Commit messages: short, imperative, no emoji.

## Folder Structure

`app/` — routes: `/` (dashboard), `/map`, `/trends`, `/alerts`, `/about`

`components/` — shared UI (SensorCard, RiskGauge, AlertItem, MapNode, etc.)

`lib/` — `sensorSimulator.ts`, `store.ts` (Zustand), `constants.ts` (node coords, thresholds)

`types/` — shared TypeScript interfaces

## Commands

Dev server: `npm run dev`

Build: `npm run build`

Lint: `npm run lint`

Always run `npm run build` before considering a feature "done" — Vercel deploys must not break.

## Deployment

Target: Vercel, connected to the GitHub repo, auto-deploy on push to `main`.

No environment variables required (no real API keys, no backend).

## What NOT to do

Don't add a real backend, database, or auth — out of scope for the demo.

Don't invent real-time hardware integration claims in UI text.

Don't use Mapbox or any service requiring an API key the team doesn't have.

Don't restructure the folder layout without flagging it first.